

## 企业实战

# 企业级文档审核 Agent 从业务问题到工程交付

不是堆功能，而是把“文档审核”  
拆成可建模、可设计、可实现、  
可部署的完整系统

★ 核心技能：把“功能清单”提升为  
“业务问题建模 + 系统交付”

## 课程设计



### 业务问题域

典型场景、竞品视角、规则配置、审核结果可解释性



### 产品交互与体验

上传 -> 处理 -> 结果 -> 反馈的审核链路，  
以及页面状态设计



### 原型需求与协同实现

Vibe Coding 输出页面结构、交互规则，  
同时锁定前后端职责边界



### 系统架构与技术选型

Agent 架构、大模型稳定性、可控性、  
可扩展性的判断



### 工程化落地

数据模型、FastAPI 接口规范、Figma -> 前端结构、  
Git 提交与 Docker 交付



最终交付：一套可演示、可联调、  
可部署的文档审核 Agent 方案

## 工程推进主线



SDD方法论深度实践

从业务问题到工程交付

Agent Team方式落地

★ 从业务出发，用工程落地，交付可用的文档审核 Agent！

# 研发方法论 · 从能力验证到产品落地

能力验证 → 工程模块 → 后端服务 → 产品型型 → 前端系统 → 联调上线

## PHASE 1 · 基础准备

1. 工具链与工作台搭建  
可观察, 可管理, 可复用

2. 系统架构抽象  
先定义模块边界 再映射组件

3. 组件调研与源码验证  
按接口反推 源码为最终证据

## PHASE 2 · 工程闭环

4. 理解沉淀为规范文档  
输入输出, 参数、依赖、产物

5. 单组件 MVP 闭环  
独立验证后再做桥接

6. 实测结果驱动  
运行时事实高于文档承诺

## PHASE 3 · 落成产品代

7. 环境隔离管理  
独立虚拟环境 防依赖污染

8. 计划驱动集成  
接口契约先行 规划驱动

9. 产品化落地  
API → 原型 → 前端 → 联调上线

## 核心研发哲学

分层拆解

证据优先

规范沉淀

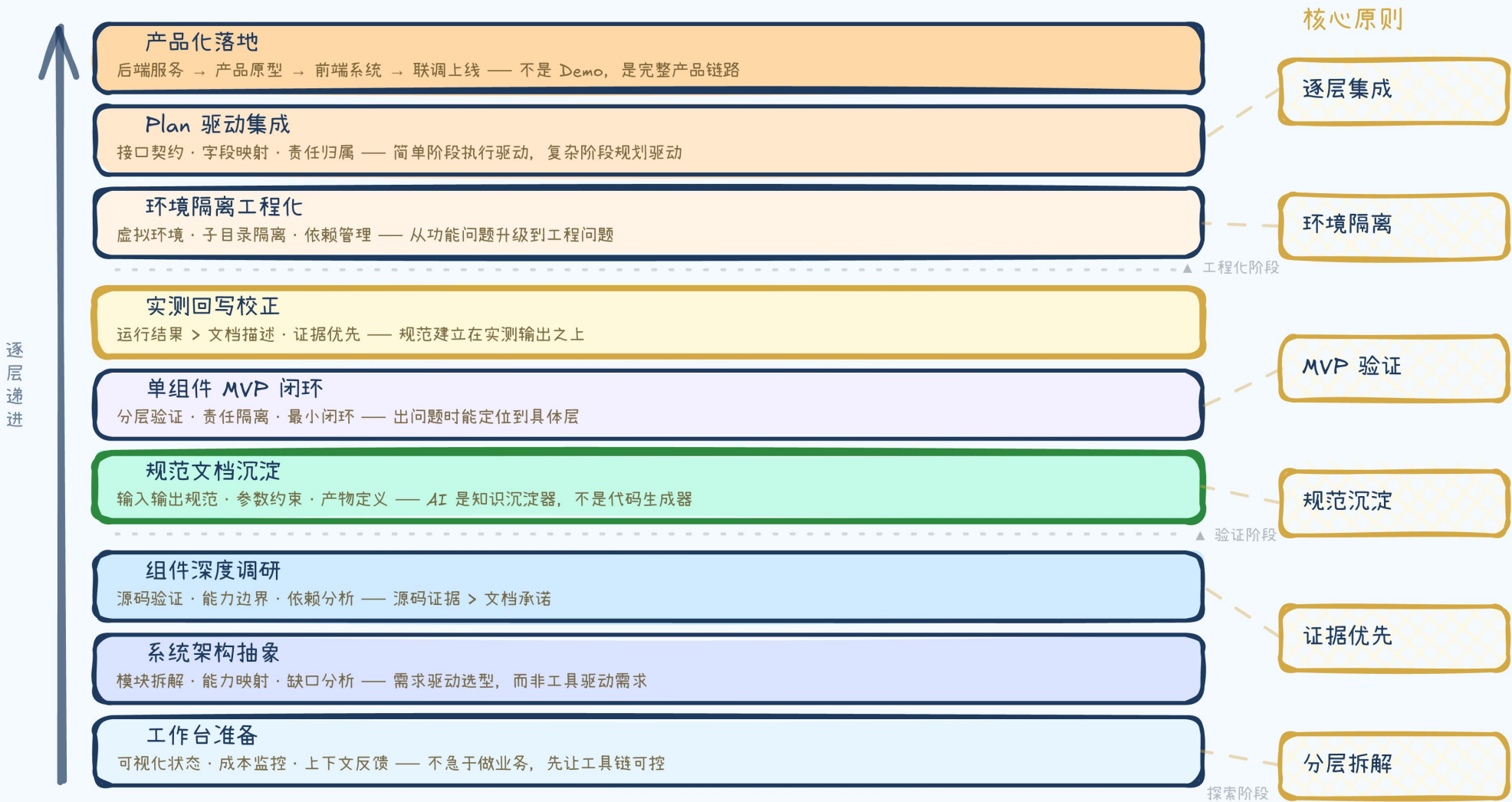
MVP 验证

环境隔离

逐层集成

先证明每个组件独立成立 → 固化接口关系 → 逐层提升为正式工程和产品

# 「证据驱动·分层递进」研发方法论



先证明每个组件单独成立 → 写清接口关系 → 提升为正式工程和产品

规范驱动  
实测回写  
分层递进



# 多智能体团队 — 分阶段协作模型



核心方法论不变，每阶段升级为「团队负责人 + 协作成员 + 任务清单 + 交付文档」

## 第一阶段

### 探索团队

- 业务场景梳理 · 竞品策略拆解
- 核心审核链路建模
- 业务状态机设计

#### 交付产出

- 《业务问题建模文档》
- 定义核心业务实体与关系
- 《竞品功能拆解报告》
- 对标分析与差异化策略
- 《审核流程与状态设计》

## 第二阶段

### 架构团队

- 智能体协作架构设计
- 规则引擎、模型工具链选型
- 数据流与服务边界划分

#### 交付产出

- 《系统架构设计文档》
- 多智能体拓扑与通信机制
- 《领域数据模型定义》
- 实体/关系/状态流转设计
- 《服务边界与接口契约》

## 第三阶段

### 构建团队

- 前端交互原型落地
- 后端 FastAPI 服务搭建
- 数据持久化 · 接口规范定义

#### 交付产出

- 《前后端工程代码骨架》
- 可运行的项目脚手架
- 《API 接口规范文档》
- RESTful 契约与数据结构
- 《前端交互设计规范》

## 第四阶段

### 集成团队

- 前后端接口对齐验证
- 全链路联调 · 异常容补齐
- 容器化封装与交付部署

#### 交付产出

- 《集成联调计划》
- 联调顺序与验收标准
- 《部署与运维规范》
- Docker 编排与环境配置
- 《可交付运行产物》



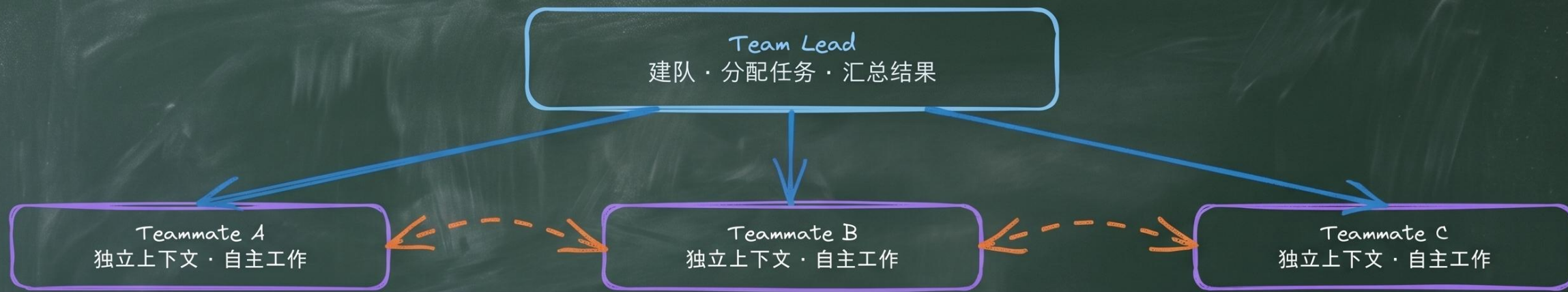
每个团队 = 团队负责人 + 协作成员 + 任务清单 + 交付文档



探索 → 架构 → 构建 → 集成 → 交付



# Claude Code Agent Teams — 多 Agent 协作架构



不是一个 Claude 开多线程，而是多个独立 Agent 协作

## 适合场景

并行研究：安全 / 性能 / 测试覆盖

新功能拆分：前端 / 后端 / 测试并行

竞争假设调试：不同队友验证不同根因

复杂物作：互相质疑、交流、对齐

## 不适合场景

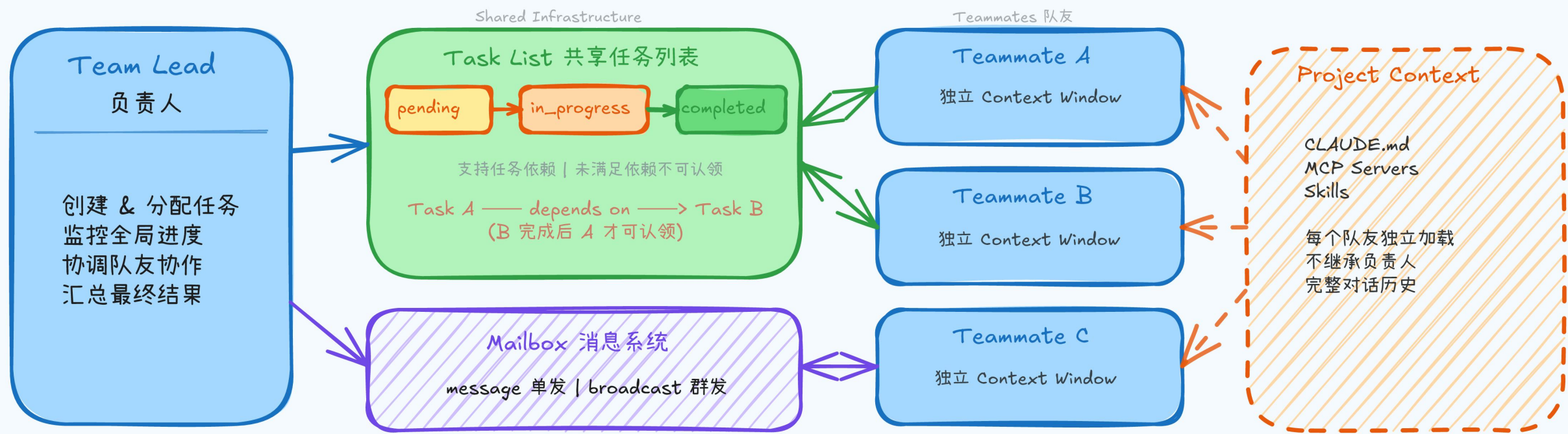
顺序依赖强的任务

多人改同一文件

小而快的任务

预算敏感的日常活

# Claude Code Agent Teams 运行原理



Core Essence

共享项目上下文，但不共享完整对话历史；共享任务状态，但保留独立思考空间。

—— 实线 = 因果 ——— 虚线 = 关联 ——

Token 成本随活跃队友数量线性增长

# Agent Teams × 研发方法论 — 智能文档审核系统

将 Agent Teams 嵌入研发全流程，而非当作「魔法开关」



▼ 拆解为 6 类并行角色

## 业务建模

- 企业审核场景分析
- 竞品拆解与对标
- 问题建模与边界定义
- 审核规则体系梳理

## 产品与交互

- 文档上传与预处理
- 处理中状态与进度
- 异常处理与降级策略
- 结果展示与反馈闭环

## 技术架构

- Agent 多步推理架构
- 规则引擎与策略层
- 可解释性输出方案
- 模型选型与 Prompt 设计

## 数据与接口

- 文档结构化数据模型
- 审核规则存储设计
- 审核结果与日志
- FastAPI 接口契约定义

## 前端实现

- 设计稿转代码落地
- 复杂交互 UI 组件
- 审核状态仪表盘
- 实时进度与通知

## 工程与交付

- 前后端联调与集成
- Git 分支与版本管理
- Docker 容器化封装
- CI/CD 自动化部署

## Agent Teams 核心使用场景

多方向研究

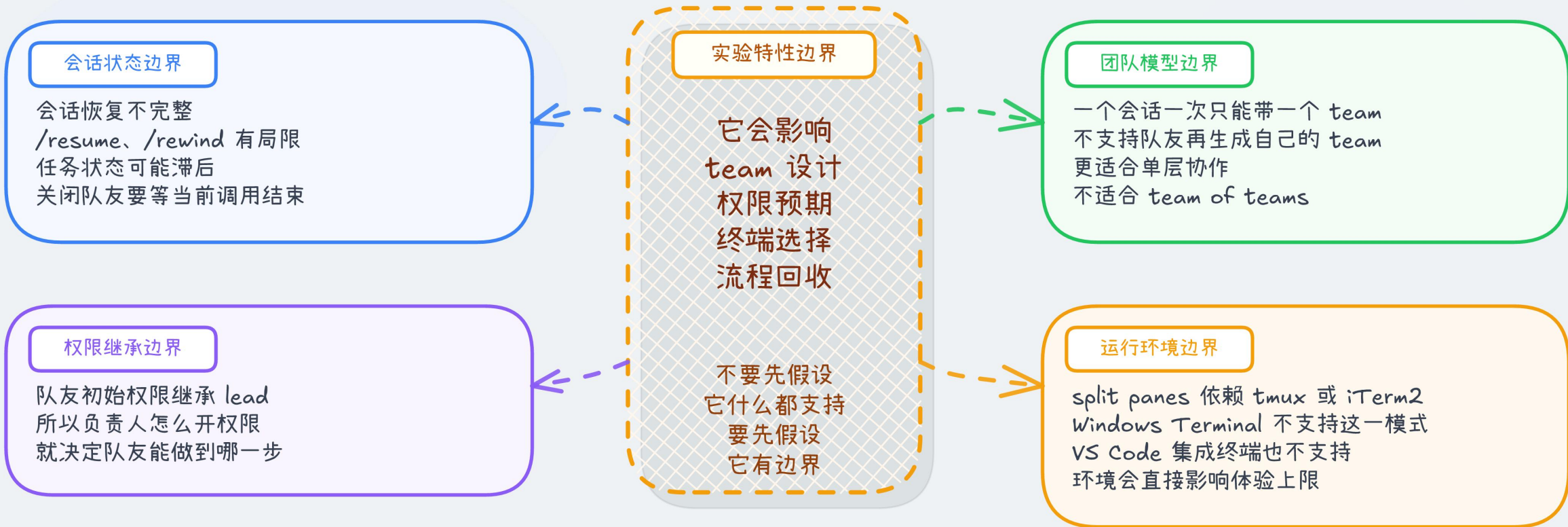
跨层协作

独立模块推进

# Agent Teams 常见问题与限制

它还是实验特性。重点不是背问题清单，而是先理解边界，再决定怎么设计和怎么用。

实验特性：边界先于能力



## 设计建议：怎么拿捏这些限制？

把它当实验特性用  
不要当默认基础设施

适合明确拆分、短周期、  
可验证的协作任务

避免强依赖 /resume、  
/rewind 的长链路设计

在 Mac/Linux + tmux/iTerm2  
环境里体验更完整

# ★ Agent Teams 实战经验分享

不是把多人同时丢出去，而是把角色、依赖、规格、门禁和记忆组织成可收敛的团队系统。

核心原则：Lead 只做编排，不做主力实现



Lead 负责定角色 / 卡依赖 / 批计划 / 收敛结果；teammates 负责干活。  
后续所有 Step 提示词都要把这条写死。



## 编排优先

### 法则一：先把团队跑顺，再谈并行规模

Lead 不是主力 coder，而是 orchestrator。真正的价值在于拆角色、写依赖、批计划、最后收敛。

Architect -> Spec Broadcast -> Backend / Frontend -> QA 这条链路非常成熟，本质仍然是先规格、后实现。

Prompt 里必须写清谁等谁开始、谁完成后通和谁、最终谁汇总，否则 team 很快会乱。



## 规格先行

### 法则二：并行不是放养，必须带 Gate

Plan approval、测试通过、错误处理、遵守 CLAUDE.md 都要前置，不要把质量留到最后补。

Agent Teams 的优势不是“多开几个 agent”，而是前后端、QA、实现者之间可以直接通值和回流。

复杂问题不要单路径推进，要引入 adversarial debugger / architecture war room 这样的对抗式分析。



## 门禁前置

### 法则三：把团队经验沉淀成基础设施

QA 不要只做一个角色。性能、可访问性，安全、响应式、坏链路要并行角色化扫描。

BLOCKED.md、PROGRESS.md、TASKS.md 要同时记录失败尝试、任务状态和推进上下文，形成共享记忆。

CLAUDE.md 在单 agent 时是加分项，在 Agent Teams 时是团队操作系统：边界、命令、禁区、约定都要写进去。



? 行动法则：什么时候  
什么时候 solo?



并行收益明显时开  
Team



需求简单、耦合很弱时，  
solo 往往更快



Team：可拆分、可并行、  
需要跨角色协同



判断标准：额外的协同成本，  
是否小于并行带来的时间收益

